

ARD2：全上下文行锚图

定义与证明资料 / 中文版

定义，2026-09-14。PDF · NER 展开器 · Python · 良序证明 · 普通 Lean 入口。

ARD2 是 **Anchored Row Diagrams (行锚图)** 的全上下文版本。一个表达式是有限列图，不是操作路径；每个关系组仍只有三个自然数坐标。与 ARD 不同，行锚和根都可以引用当前列，根也不必在父列之前。软件资源保护不是数学规则。

1. 有限语法与根闭包

有限图是列串 $G = (C_0, \dots, C_{m-1})$ 。第 j 列中的压缩组为

$$(k, p, q), \quad 0 \leq k, q \leq j, \quad 0 \leq p < j.$$

三个坐标依次是**行锚**、**父列**、**最大根**；子列由该组所在的第 j 列确定。此组表示所有 $0 \leq t \leq q$ 的原子关系 (k, t, p, j) 。这是根闭包，不是行闭包。同一 (k, p) 只保留最大的 q ，重复关系没有影响。空列仍是表达式的一部分。

$k = j$ 和 $q = j$ 分别是行 SELF 与根 SELF：它们表示这个特定子列的地址，不是固定的全局常量。复制时，两者都必须随子列一起搬运。父列不能是 SELF。不要求 $k \leq p$ 或 $q \leq p$ 。

零图没有列，写作 $\emptyset$ 。一个空列 $[]$ 表示一； m 个空列表示自然数 m 。无损列表每列使用一对方括号，列内三元组以逗号分隔：

```
[ ][(1,0,1)][(2,1,2)]
```

这是三列种子 A_3 。Python 与 NER 保存最大根；Lean 有限几何使用显式根闭包的关系。两种编码的对应包含规范化与去重。

2. 按列比较

每列的压缩组按 (p, k, q) 递减排序，再按相同键对组列表作字典序比较。图从最左列开始作字典序比较。两个层级都规定真前缀较小。只需最早不同列及其中最早不同组；比较不运行展开，也不搜索路径。

将全部根显式展开并按 (p, k, t) 排序，得到相同的顺序。对固定 (p, k) ，最先不同的最大根会在到达后面的组之前决定比较。

另加最大外顶端 Top，显示为 Limit of ARD2。它不是有限列，也不能用作坐标。

3. 基本列

指标 n 是非负整数。规定 $0[n] = 0$ ；这个恒等式不算严格下降步骤。对非空图令 $x = m - 1$ ，以 ∂G 表示删掉末列，并规定

$$G[0] = \partial G, \quad G[n] = \partial G \quad \text{若 } C_x = \emptyset, \text{ 则对所有 } n \text{ 成立。}$$

其余情形满足 $n > 0$ 且末列非空。按**行**、**根**、**父**的键 (k, q, p) 选择末列最大组，记为 (K, c, r) 。控制顺序与比较时的**父**、**行**、**根**顺序不同。置

$$L = x - c > 0, \quad N_b = x + bL, \quad \phi_b(i) = \begin{cases} i, & i < c, \\ i + bL, & i \geq c. \end{cases}$$

切点是控制父 c 。输出有 $x + nL$ 列。取以下各组的并集，再规范化：

1. 切点 c 以前的前缀只保留一份。对 $b = 0, \dots, n$ 复制来源列 $C_c, \dots, C_{x-1}$ ，将第 j 列的 (h, p, q) 移到第 $\phi_b(j)$ 列的 $(\phi_b(h), \phi_b(p), \phi_b(q))$ 。
2. 对每个 $b < n$ ，在第 N_b 列放接缝。旧末列组 (h, p, q) 的贡献为：
 - 若 $h < K$ ，加入 $(\phi_b(h), \phi_b(p), \phi_b(q))$ ；
 - 若 $h = K$ ，加入 $(\phi_b(h), \phi_b(p), \min(\phi_b(q), \phi_b(r) - 1))$ ，但最后一个坐标非负时才加入；
 - 若 $h > K$ ，不加入。
3. 在同一接缝，加入所有 $(h, \phi_b(c), N_b)$ ，其中 $0 \leq h < \phi_b(K)$ 。

来源块共有 $n + 1$ 份，接缝有 n 个。新复制块的首列与其前面的接缝占据同一位置，因此需要合并各组。等价地，也可在每个块中复制 x 以前的整个前缀：切点 c 以前的各份复制完全重合。

新生低行包的最大根是 N_b ，**即接缝自身**，不是 $\phi_b(c)$ 。来源复制的四个坐标全部搬运。特别是来源切点列中的行 SELF 或根 SELF，要重新绑定到它被复制进去的接缝列。

最大根搬运后应取完整根闭包，其中包括移动映射跳过的中间根。若只搬运旧的单个根而不重新闭包，就变成了另一套规则。每次调用只有有限循环，不递归展开自身的输出。

令 $H = A_2$ ，前几项为

```
H      = [[(1,0,1)]
H[0]   = []
H[1]   = [[(1,0,0),(0,0,1)]
H[2]   = [[(1,0,0),(0,0,1)][(2,1,1),(1,1,2),(0,1,2)]
```

4. 种子、标准域与两种良基结论

种子 A_n 有 n 列，其中

$$C_0 = \emptyset, \quad C_j = \{(j, j-1, j)\} \ (j > 0), \quad \text{Top}[n] = A_n.$$

因此 $A_0 = 0$ 、 $A_1 = []$ ，且 $A_{n+1}[0] = A_n$ 。令 $D(H)$ 包含 H 及其有限次展开后代，忽略 $0 \rightarrow 0$ 自环。有限标准域和完整记号分别为

$$U = \bigcup_{n < \omega} D(A_n), \quad U \cup \{\text{Top}\}.$$

标准性就是从这些种子经过有限展开可达，不用可及性或语义表示存在性定义。构造器或解析器只检查结构合法性，不保证任意输入属于 U 。

展开保持合法性。对每个有限图 G ，旧末列以前的全部列保持不变，且 $G[n]$ 是 $G[n+1]$ 的完整列前缀；若旧末列非空，则是真前缀。从非零图出发的每一步展开，都严格降低指定列序。

纸面证明 与普通 Lean 入口 区分以下两项结论：

- **全部合法有限图**：非零展开关系良基。纸面证明构造每次展开严格下降的序数秩；Lean 给出 `valid_step_wellFounded`。
- **标准域**：指定列序在 U 上良序，加入最大外顶端后仍良序。Lean 给出 `standard_strictWellOrder`、`standard_with_top_strictWellOrder`，以及纸面定义标准域版本 `paper_standard_with_top_strictWellOrder`。全部合法图的列序**并不良序**。例如

$$[\emptyset]^{r+1}[(0,0,0)] > [\emptyset]^{r+2}[(0,0,0)] > \dots$$

是合法图的列序降链，但不是展开链。第二项结论中的标准域限制不能删去。

纸面公理上界为 KP_ω + “存在不可数序数”，包含完整集合归纳，不包含幂集或选择公理。实际普通 Lean 证明已经编译成功，最终公理报告只使用 `propext`、`Classical.choice`、`Quot.sound`。这~~不是~~在 Lean 中编码该弱体系的语法和内部推导。定义一致性模块 将纸面有限规则及其可达域与 Lean 定义连接起来，但并不验证 Python 或 JavaScript 虚拟机。编译与验证状态见仓库验证记录。

这里没有证明 ARD2 与 ARD、RPD、Y、wY 或 IPD 之间的序型不等式。“ARD2”这个名称、共用证明方法及更大的有限计数，都不能证明它对应更大的序数，也不能证明前序段关系。不声称上述公理上界最优。

5. 局部计数与五种显示

固定某一列之前的前缀，将它看作末列；作一次正指标展开后，仅丢弃新追加的列。重复操作，直到保留位置的列为空，再将其删除。该列的**局部计数**包含最后一次删除，所以空列计数为一。保留部分与选用哪个正指标无关。非空末列被改写后，其局部计数恰好减一。

在位置 j 的局部一步中，来源切点列 C_c 合入保留的接缝：来源行 c 和来源最大根 c 都重新绑定为 j ，来源父仍固定在 c 之前。生成低行包的最大根为 j 。这与旧 ARD 计数算法使用的前提不同。

固定前缀与末位置 j 后，规范末列至多有 $(j+2)^{j(j+1)}$ 种状态：共有 $j(j+1)$ 个可能的 (k, p) ，每对选择缺省或最大根 $0, \dots, j$ 。每次非空局部操作严格降低这个有限列序，因此局部计数必终止，无需模拟整个反复 [1] 的轨道。种子 $A_1, \dots, A_5$ 的计数序列依次是

```
1, 5, 55, 969, 23751
```

的各个前缀。这些是精确核对的数值，不声称闭式公式。计数只是显示，不是无损输入格式或比较键；不提供从计数反向解析的功能。

文件恰有以下五种显示，不重复添加“列表”按钮：

1. **列表**：默认，无损、单行的三元组列表。
2. **计数序列**：使用 `BigInt` 的精确局部计数。
3. **弧线图**：按行锚分层，显示全部关系组。每条弧线连接子列与父列，带外圈数字表示最大根；红点标出该行锚所引用的列。轨道高度只用于走线，不是额外数学层级。
4. **邻接表（文字）**：每列一对 `[]`；分号从行零起分层，逗号从父列零起定位，格内填写最大根。保留中间空位，省略尾部空位；数字 0 不代表空位。例如 A_2 显示为 `[][];1]`。该视图可逆。
5. **邻接表（图）**：每个实际使用的行对应一张完整、紧凑的上三角表，最上方另放一行计数序列。浅色对角格填写列号，同时充当行标和列标；不再画外侧坐标标签。左侧是裸行锚数字，其余格内填最大根。由于父 $p < j$ ，被省略的下三角确实没有关系，即使行或根是 SELF 也不例外。

列表与计数字体继承页面普通字体。图不会裁出局部并冒充完整图。计数使用隔离的约一秒及工作量预算：若耗尽，明确不显示计数；只要几何本身可画，仍显示全部关系。若几何本身超限，则显示警告，不画局部图。资源超限提示既不是近似数值，也不是数学不终止的证据。

6. 软件使用与检验

在 NER 的自定义记号入口导入 `ARD2.ne-rewritten.js`，然后选择 **ARD2**。注册 ID 为 `ard2-v01`，默认显示为 列表；“等价表示”菜单提供上述另外四种视图。不需要依赖脚本、网络访问或持久缓存。

主要输入支持 `A3`、`A3[2][1]`、`Limit[3]`、`Limit of ARD2`、自然数以及完整三元组列表；忽略空白。自然数表示相应数量的空列，不是计数序列编码。NER 的 `FS`、`FS_alter`、`FS_short` 三个选项使用完全相同的规则，不偏移指标。

将 Python 文件放入模块搜索路径，使用 Python 3.10+：

```

from ard2 import ARD2, ZERO, TOP

a = ARD2.seed(3)
assert str(a) == "[[(1,0,1)][(2,1,2)]"
assert TOP[3] == a
assert a[0] == ARD2(a.columns[:-1])
assert a[1] < a
assert a[1].columns == a[2].columns[:len(a[1].columns)]
assert a.local_step().columns == a[1].columns[:len(a.columns)]

```

ARD2() 和 ZERO 是零；ARD2.finite(n) 有 n 个空列；TOP 为 ARD2(None)。构造器检查坐标、合并重复组并规范化各列。kind 为 zero、successor 或 limit；有限非零式当且仅当末列为空时为后继。fs(n) 与 $[n]$ 相同。local_step() 返回一次固定前缀的局部步骤，而非整个计数；它拒绝外顶端。自然数输入拒绝负数、布尔值和非整数。

Python 定义仅依赖标准库，整数任意精度。它有意省略解析、完整计数器、绘图、缓存与资源保护。大指标可能耗尽时间或内存，实验需要自行设置上限。

独立的 Python 测试、NER 规则测试和显示测试 覆盖完整根闭包、两种 SELF 重绑、低行生成、前缀与比较行为、精确小计数和完整显示。测试明确限制时间、宽度和内存，不代替良序证明。在仓库根目录运行：

```

python -B tests/test_ard2.py
node --max-old-space-size=256 tests/ard2_ner.cjs
node --max-old-space-size=256 tests/ard2_display.cjs

```